

Qwen3-Coder-Next Tier 3

Part 14 Detailed Implementation Specification

Deployment & Human Approval

Engineering specification for the subsystem that decides when software is allowed to leave the lab, who must bless that decision, and how the system proves it did not smuggle a bad release past the gate.

Document role	Implementation specification
Part	14
Subsystem	Deployment & Human Approval
Dependency posture	Consumes benchmark verdicts, evaluation outputs, and release artifacts; publishes promotion and rollback decisions

Scope note: this document explains how to implement controlled promotion, approval capture, rollout, and rollback. It does not redesign the architecture. It does not replace human decision-making with a louder button.

1. Purpose

The Deployment & Human Approval subsystem exists because an autonomous coding system should not be allowed to turn every successful run into a production push. The system needs a controlled boundary between 'generated something plausible' and 'this artifact may now affect real users, data, or money'.

This part solves the last-mile problem of autonomy. Earlier parts can plan, research, implement, test, review, evaluate, recover, compare, and benchmark. None of that matters if a bad build can still slip into a live environment without a traceable decision. Deployment is therefore a gated process, not a reflex.

Future parts depend on this boundary. The final integrated system, release automation, and human approval workflows all need one source of truth for promotion state, decision provenance, and rollback eligibility.

Principle: the system may recommend, stage, and verify. It may never silently promote itself.

2. Scope

Included

- Deployment request assembly from release-ready artifacts.
- Environment target resolution for local, staging, canary, and production channels.
- Policy and gate checks that combine benchmark verdicts, evaluation outputs, and explicit release rules.
- Human approval request generation with a clear decision payload and traceable evidence bundle.
- Decision ingestion and state transition management for approve, reject, hold, and rollback outcomes.
- Rollout orchestration, verification, and promotion finalization.
- Rollback orchestration and failure-safe abort paths.
- Immutable audit records for every decision, artifact, and environment transition.

Excluded

- Build and inference internals, which are owned by earlier parts.
- The benchmark harness itself, which already produces verdicts and trend data.
- The detailed design of any external approval user interface beyond the interface contract needed to receive and store a decision.
- Organization-specific infrastructure topology outside the environment abstraction layer.
- Business policy content that belongs to a release manager or security team rather than the agent system.

Included	Excluded
Artifact promotion control	Build / inference implementation
Approval request and decision records	Benchmark metric calculation
Controlled rollout and rollback	Business-specific release policy authoring
Audit trail and decision provenance	External UI implementation details

3. System Overview

High-level workflow

```
User / CI / Release Manager
-> Deployment Request
-> Release Orchestrator
-> Artifact Resolver
-> Policy and Gate Engine
-> Approval Packet Builder
-> Human Approval Layer
-> Decision Ingestor
-> Rollout Manager / Rollback Manager
-> Environment Verification
-> Promotion Record / Audit Log
```

A deployment request starts with a release candidate, an environment target, and a reason that can be read by a human after the fact. The orchestrator resolves the exact artifact, checks it against policy, and stops if the release does not qualify.

If policy passes, the system builds an approval packet. That packet contains the artifact identity, benchmark verdicts, recent regressions, environment constraints, and the specific consequence of approving the release. Humans approve or reject the packet; the system records the decision, not the narrative.

After approval, the rollout manager performs the deployment in the selected environment and verifies the target state. If verification fails, the rollback manager returns the system to a known-good state.

Conceptual example

```
Release candidate: rc-2026-14-0081
Environment: staging -> canary -> production
Policy result: benchmark PASS, evaluation PASS, approval REQUIRED
Decision: HUMAN APPROVED
Rollout: canary success, production promoted
Audit: immutable record written with artifact hash, approver ID, timestamps, and rollback eligibility
```

The approval layer is not decorative. It is the point where automation stops and accountability starts.

4. Internal Architecture

This subsystem is intentionally narrow. It must separate decision logic, approval capture, rollout control, and audit storage so one failure does not erase the evidence of another.

Component	Responsibility	Primary failure mode
Release Orchestrator	Owns the deployment lifecycle, state transitions, and idempotent command handling.	Duplicate release, skipped state, broken lifecycle
Artifact Resolver	Binds a deployment request to the exact artifact bundle and immutable identifiers.	Wrong artifact, hash mismatch, stale bundle
Policy and Gate Engine	Evaluates benchmark verdicts, environment rules, and release thresholds.	Bypassable gate, missing rule, incorrect allow/deny
Approval Packet Builder	Produces the human-readable evidence bundle and approval request payload.	Incomplete evidence, unreadable packet, wrong target
Decision Ingestor	Records human approval, rejection, hold, or rollback decisions with provenance.	Lost decision, ambiguous actor, tampered record
Rollout Manager	Executes the deployment plan and monitors post-deploy verification.	Partial rollout, silent failure, unobserved drift
Rollback Manager	Restores the previous known-good state and closes the incident path.	Failed rollback, inconsistent environment, incomplete revert
Audit Store	Persists immutable decision and rollout history.	Missing lineage, overwritten evidence, unreadable trace

Required abstractions: `DeploymentRequest`, `DeploymentPlan`, `ApprovalPacket`, `ApprovalDecision`, `RolloutState`, `VerificationResult`, and `AuditEvent`. These are the contracts that downstream systems will consume without needing to know how the release was executed.

Runtime interaction diagram

```
Deployment Request
-> Orchestrator
-> Artifact Resolver
-> Policy Engine
-> Approval Packet
-> Human Approval Layer
-> Decision Ingestor
-> Rollout Manager
-> Verification
-> Audit / Promotion Record
```

Design rule: policy decides whether approval is required or blocked; humans decide whether a required release may proceed. The code must never silently downgrade a required approval into a soft suggestion.

5. Project Structure

Recommended directory structure below. It separates orchestration, policy, approval, rollout, rollback, and storage so a future engineer can locate responsibility without reading half the repository.

Directory	Purpose
deployment/	Top-level release orchestration, release state, and deployment flow control.
deployment/requests/	Schemas and parsers for deployment requests, plans, and environment targets.
deployment/policies/	Gate rules, threshold evaluation, and release qualification logic.
deployment/approval/	Approval packet builder, decision ingestion, and decision history handling.
deployment/rollout/	Controlled rollout executor, verification, and promotion finalization.
deployment/rollback/	Known-good restore logic, rollback policies, and abort handling.
deployment/audit/	Immutable release history, trace events, and compliance export helpers.
deployment/storage/	Persistence adapters for release state, artifacts, and decision records.
config/	Environment matrices, policy thresholds, approval settings, and rollout profiles.
schemas/	Versioned contracts for requests, packets, decisions, and outcomes.
tests/	Unit, integration, approval simulation, and rollback drill fixtures.

The release subsystem should live beside benchmark and evaluation code, not inside the product deployment target. That keeps the control plane separated from the thing being controlled.

6. Data Contracts

Input structures

Required fields	Notes
request_id, artifact_id, artifact_hash, suite_verdict_ref, environment_target, initiator, reason	Must be replayable and traceable.
environment_name, channel, constraints, secret_scope, rollback_policy	Defines what may change and how.
plan_id, steps, prechecks, approval_required, verification_policy	Single release plan for one target.
packet_id, request_id, evidence_bundle, benchmark_refs, risk_notes, approval_deadline	Human-readable plus machine-readable.

Output structures

State should be explicit and finite. Suggested lifecycle: draft, gated, awaiting_approval, approved, deploying, verifying, promoted, rejected, rolled_back, failed, cancelled. Each transition must be represented in storage and visible in audit records.

Configuration format

- Approval mode: required, optional, or bypass-locked for a channel.
- Environment profile: local, staging, canary, production, or custom.
- Rollback policy: automatic, manual-only, or approval-required.
- Verification window: health-check duration and success criteria after rollout.
- Evidence policy: which benchmark and evaluation artifacts must accompany a request.

7. State Management

The deployment layer owns release state. The rest of the system may request promotion, observe state, and consume final records, but it may not rewrite deployment history. A release without lineage is just a rumor with timestamps.

State category	Owner	Persistence model	Consumed by
Deployment requests	Release orchestrator	Append-only request store	Policy engine, approval layer, audit tools
Approval packets	Packet builder	Immutable packet archive	Humans, compliance, replay tools
Approval decisions	Decision ingestor	Append-only decision log	Rollout manager, reports, audit
Rollout progress	Rollout manager	Lifecycle event stream	Verification, rollback, status display
Promotion records	Orchestrator	Immutable release record	Release managers, dashboards, downstream automation
Rollback traces	Rollback manager	Incident and recovery log	Postmortem tools, future gates

State changes must be monotonic whenever possible. Once a release is promoted or rolled back, the outcome is frozen. If later analysis is needed, publish a derived audit artifact rather than mutating the original record.

Future agents consume this state as evidence. The benchmark harness can answer whether a build was eligible; the evaluator can explain why it was risky; the final integrated system can decide whether the same class of release should be blocked in future. Nobody should have to read deployment logs line by line to know what happened.

8. Component Specifications

Release Orchestrator

- Purpose: own the end-to-end deployment lifecycle and guard transitions.
- Inputs: deployment request, environment target, policy result, approval decision.
- Outputs: release plan, lifecycle state updates, promotion or rollback command.
- Failure conditions: duplicate request, invalid state transition, incomplete evidence bundle.
- Future integration points: final release automation and human override policy.

Artifact Resolver

- Purpose: bind the request to an immutable artifact bundle and hash.
- Inputs: build artifact reference, revision, release channel, retention policy.
- Outputs: resolved artifact identity and verification digest.
- Failure conditions: stale artifact, hash mismatch, missing provenance.
- Future integration points: supply-chain checks and provenance export.

Policy and Gate Engine

- Purpose: determine whether deployment requires approval, is blocked, or may proceed to packet creation.
- Inputs: benchmark verdicts, release thresholds, environment rules, exceptions.
- Outputs: gate result and reason codes.
- Failure conditions: ambiguous thresholds, missing benchmark evidence, policy drift.
- Future integration points: release governance and channel-specific policy packs.

Approval Packet Builder

- Purpose: produce a concise, reviewable evidence bundle for humans.
- Inputs: request data, artifact identity, gate result, benchmark summaries, diff notes.
- Outputs: approval packet and evidence references.
- Failure conditions: unreadable summary, missing artifact ref, incomplete provenance.
- Future integration points: approval UI or external ticketing system.

Decision Ingestor

- Purpose: record human approval or rejection with provenance and timestamps.
- Inputs: approval packet ID, actor identity, decision, rationale, optional comments.
- Outputs: immutable decision record and next-state command.
- Failure conditions: unverified actor, mismatched packet ID, duplicate decision write.
- Future integration points: compliance audit and release analytics.

Rollout and Rollback Managers

- Purpose: execute promotion, verify health, and recover from failure.
- Inputs: approved deployment plan, environment handles, rollback policy, verification window.
- Outputs: rollout status, verification result, rollback trace, final release status.
- Failure conditions: partial rollout, failed verification, incomplete rollback, environment drift.
- Future integration points: incident response and future automation policy.

9. Implementation Sequence

Step 1 - Define release schemas and lifecycle states

Create the contracts first so every downstream module uses the same request, decision, rollout, and audit vocabulary. Output: versioned schemas, lifecycle enum, validation rules. Dependency note: nothing else should be written before the contracts exist.

Step 2 - Implement persistent request, decision, and audit storage

Add append-only stores and retrieval helpers for release evidence. Output: request store, decision log, audit store, index lookups. Dependency note: depends on schemas.

Step 3 - Implement artifact resolution and policy evaluation

Bind the request to a real artifact and decide whether the release may proceed. Output: artifact resolver and policy engine. Dependency note: depends on schemas and storage.

Step 4 - Build approval packet generation

Assemble the evidence humans must review. Output: approval packet builder and evidence bundle serializer. Dependency note: depends on gate results and artifacts.

Step 5 - Build decision ingestion and state transitions

Store the human decision and move the release through explicit lifecycle states. Output: decision ingestor and state machine transitions. Dependency note: depends on packet generation.

Step 6 - Build controlled rollout and verification

Execute deployment to the target environment and verify health before promotion. Output: rollout manager and verification adapter. Dependency note: depends on approved decisions.

Step 7 - Build rollback and abort paths

Ensure failures revert safely and leave a trace. Output: rollback manager and failure-safe abort behavior. Dependency note: depends on rollout control.

Step 8 - Build audit reporting and release summaries

Generate release records suitable for dashboards, audits, and human review. Output: promotion record and audit export. Dependency note: depends on final state.

Step 9 - Add simulation, approval, and rollback tests

Validate the full lifecycle with positive and negative fixtures. Output: reproducible end-to-end tests. Dependency note: depends on the complete path.

10. Validation Strategy

Validation must prove that the deployment layer respects policy, records human decisions correctly, and leaves the system in a known state after success or failure. A working demo is not proof. A reproducible release record is proof.

Test layer	What it verifies	Success condition
Unit tests	Schema validation, lifecycle transitions, policy rule evaluation	Pure functions produce expected outputs from fixed inputs.
Integration tests	Request -> approval packet -> decision -> rollout -> record	A synthetic release completes with correct artifacts and state.
Approval simulation tests	Decision capture and reviewer provenance	Approvals and rejections are stored exactly as issued.
Rollback drills	Failure recovery and state restoration	A failed rollout returns to the prior known-good state.
Replay tests	Deterministic request and decision reconstruction	The same request and evidence set yield the same release decision.
Audit tests	Immutable history and lineage preservation	No release record is rewritten after finalization.

Validation must also check negative cases. A missing benchmark verdict must block release. A stale artifact must block release. An approval packet without human approval must not proceed. A failed verification must trigger rollback, not optimism.

Important: if a release completes but the audit record cannot prove what was approved, by whom, for which artifact, and against which policy version, the release is not acceptable.

11. Deliverables

Artifact	Description
Release orchestration module	Coordinates the full deployment lifecycle and state transitions.
Artifact resolution module	Maps deployment requests to immutable artifact identities and hashes.
Policy engine	Evaluates benchmark and release rules before approval or rollout.
Approval packet builder	Produces the evidence package a human reviewer can inspect.
Decision ingestion module	Stores approval, rejection, hold, and rollback decisions with provenance.
Rollout controller	Executes staged deployment and post-deploy verification.
Rollback controller	Restores the last known-good state and records the recovery path.
Audit store and export	Immutable history of requests, decisions, rollouts, and outcomes.
Config templates	Environment, gate, and rollback settings by release channel.
Test fixtures	Approval and rollback scenarios for repeatable regression testing.

Deliverables must be independently deployable and importable. If an artifact cannot be consumed by another module or validated in isolation, it is not finished; it is a script wearing a fake moustache.

12. Acceptance Criteria

Criterion	How it is verified
A release cannot be promoted without passing policy checks.	Integration test attempts promotion with failing benchmark or policy evidence and expects a block.
Every human approval or rejection is recorded with provenance.	Approval simulation verifies actor, time, packet ID, and rationale are stored.
Rollout state is visible and explicit at every stage.	State transition test confirms draft, gated, awaiting_approval, approved, deploying, verifying, promoted, rejected, rolled_back, failed, or cancelled states are reachable and logged.
Rollback returns the system to a known-good state.	Rollback drill confirms prior artifact remains active and recovery record exists.
Audit records are immutable.	History replay test confirms finalized records are not overwritten.
Release records can be reproduced from stored evidence.	Replay test reconstructs the same decision from the same request and policy version.
A failed verification does not become a silent promotion.	Negative integration test forces verification failure and expects rollback or fail state.

Completion means that a release manager, an engineer, and the automation itself can all answer the same question from the same artifacts and get the same answer. If the answer changes depending on who asks, the subsystem is not done. It is merely theatrical.

13. Common Failure Modes

- Treating approval as a suggestion instead of a required gate.
- Coupling the approval record to mutable deployment state so evidence changes after the fact.
- Letting benchmark failures be overridden without an explicit policy path.
- Recording the rollout status but not the exact artifact identity that was deployed.
- Allowing rollback to succeed operationally while losing the audit trail.
- Mixing environment policy with app-specific business logic and making the release layer brittle.
- Using a human approval UI as the source of truth instead of the signed decision record.

The most dangerous trap is 'soft approval'. If the system can implicitly proceed because the reviewer was slow, the gate is fake. The second danger is trace decay: the release is technically recoverable, but nobody can prove what happened. That is still a failure.

14. Future Integration

- Part 15: near-Codex integrated system, where this layer becomes the final promotion boundary.
- Part 13: benchmark harness verdicts and trend data become required release evidence.
- Part 12: multi-agent coordination metrics can inform rollout risk and promotion confidence.
- Part 8: failure recovery provides the rollback and safe-abort logic used during deployment failures.
- Part 7: evaluator scoring helps the policy engine judge whether a candidate is fit to be released.

The integration rule is one-way: upstream parts publish evidence and downstream consumers make release decisions from that evidence. Deployment must not become a dumping ground for runtime analysis that belongs elsewhere.

Bridge to the final system: when all earlier parts are in place, this component becomes the single controlled exit path from autonomous work into live environments.

15. Completion Checklist

- Release requests are schema-validated and versioned.
- Artifact identity and hash are recorded before approval.
- Policy checks block or require approval exactly as configured.
- Approval packets include the evidence needed for a human decision.
- Human decisions are ingested and persisted with provenance.
- Rollout and rollback flows are both testable and auditable.
- Verification failures do not promote releases.
- Audit records are append-only and immutable.
- Promotion records link the request, artifact, decision, and environment.
- Simulation fixtures cover approve, reject, hold, deploy, verify-fail, and rollback paths.

The subsystem is complete when a release can be explained end-to-end from request to final status without relying on memory, browser history, or a heroic engineer who remembers what happened at 2 a.m. The evidence must stand on its own.